

Introduction to Transformers

Transformers

LLMs are built out of transformers

Transformer: a specific kind of network architecture, like a fancier feedforward network, but based on attention

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

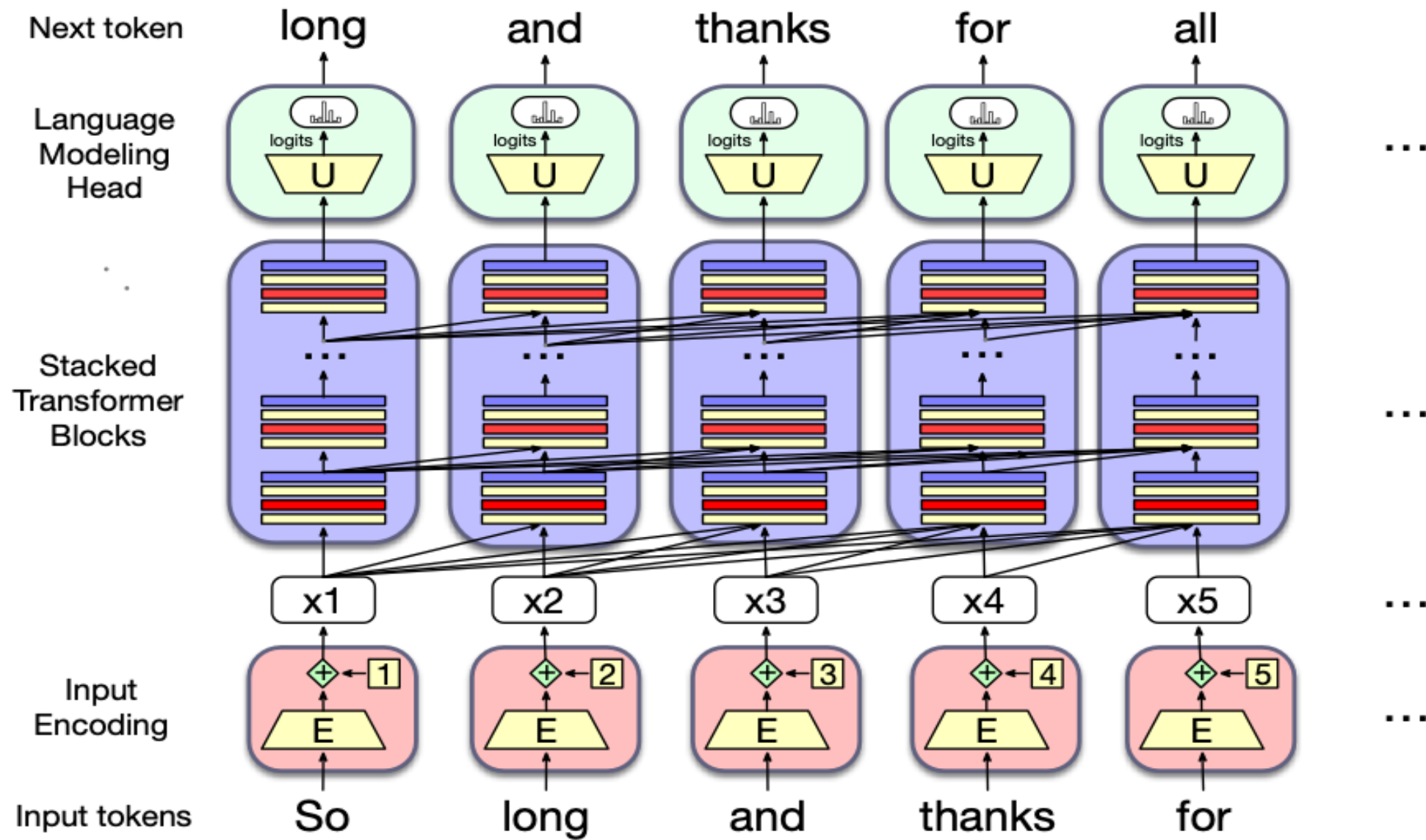
Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

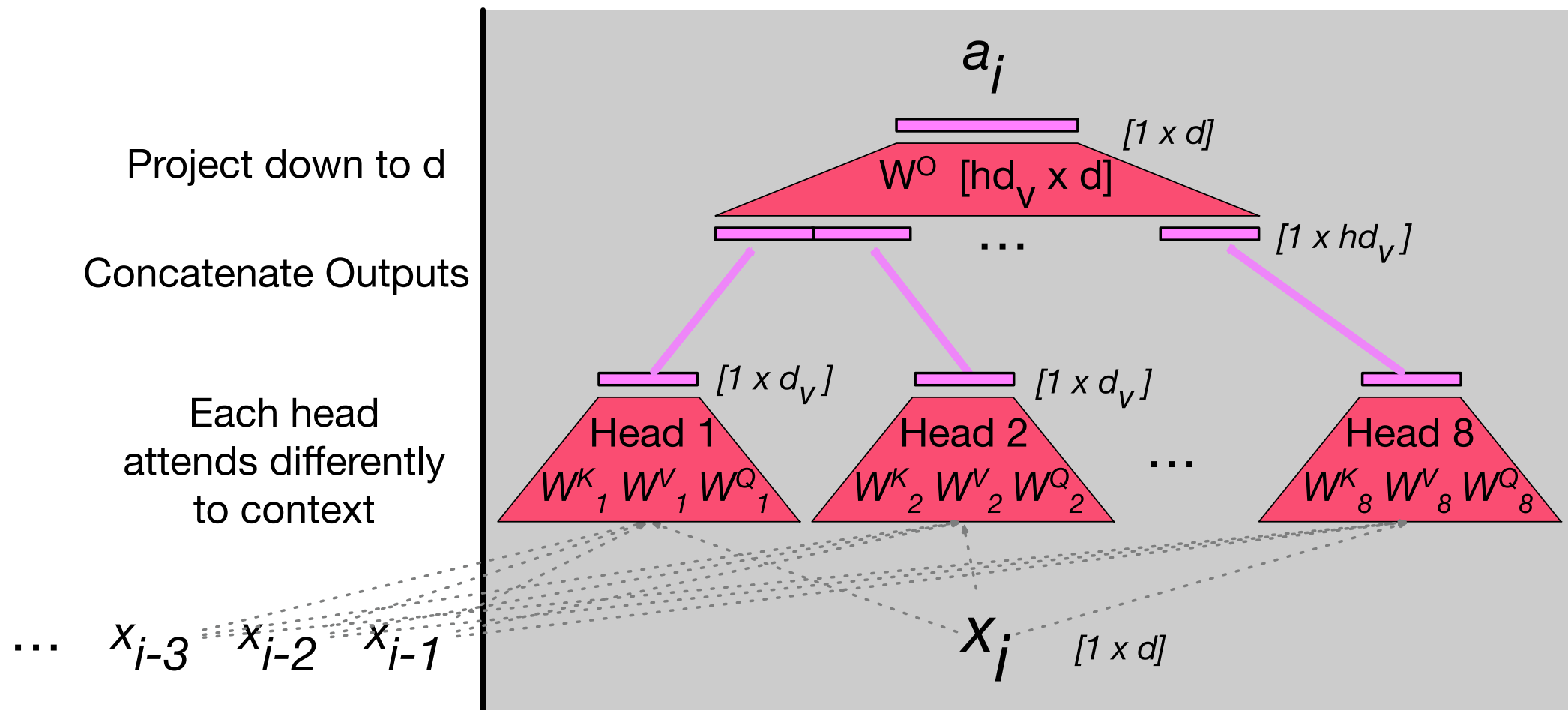
Attention

Transformers

Instead of starting with the big picture

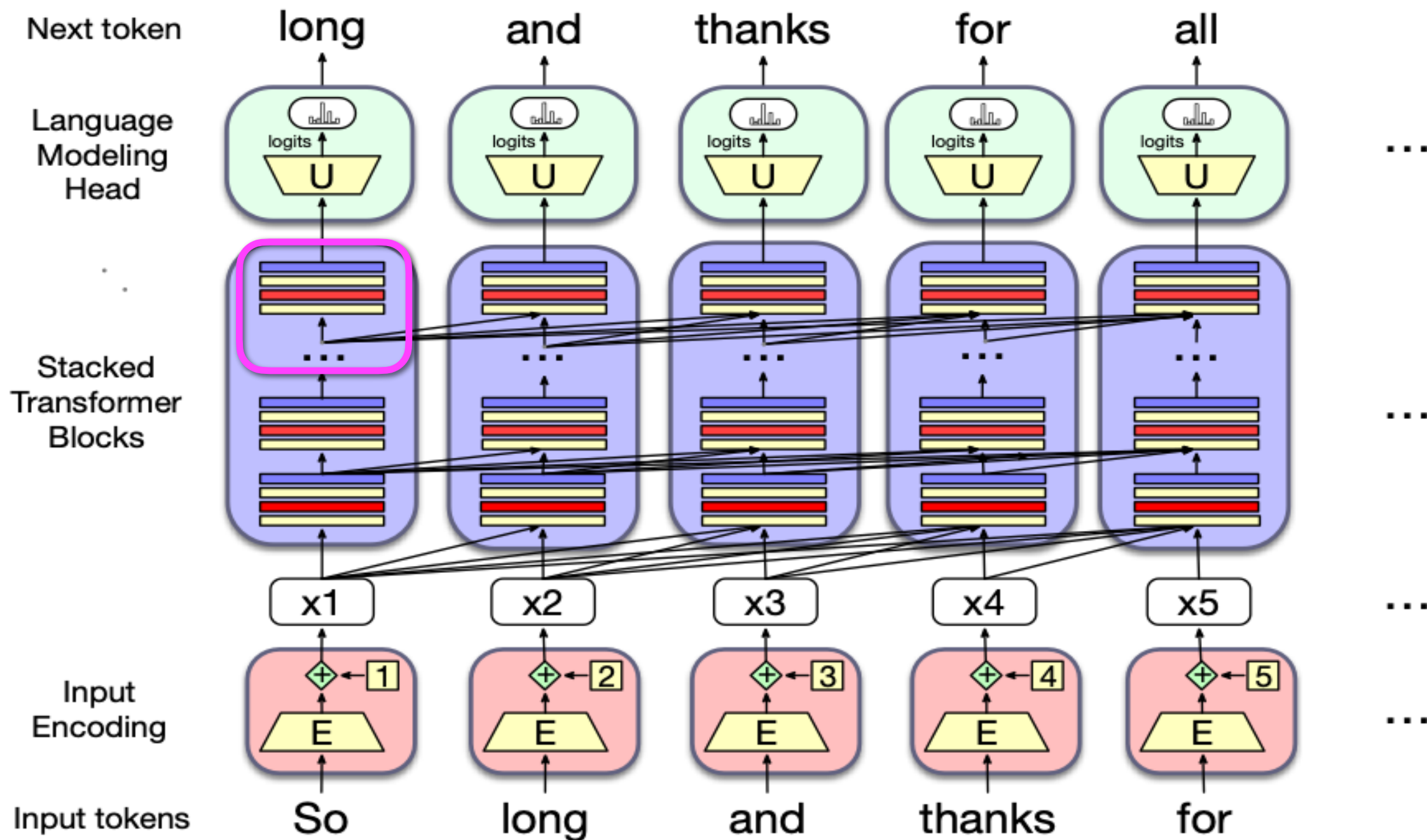


Multi-head attention

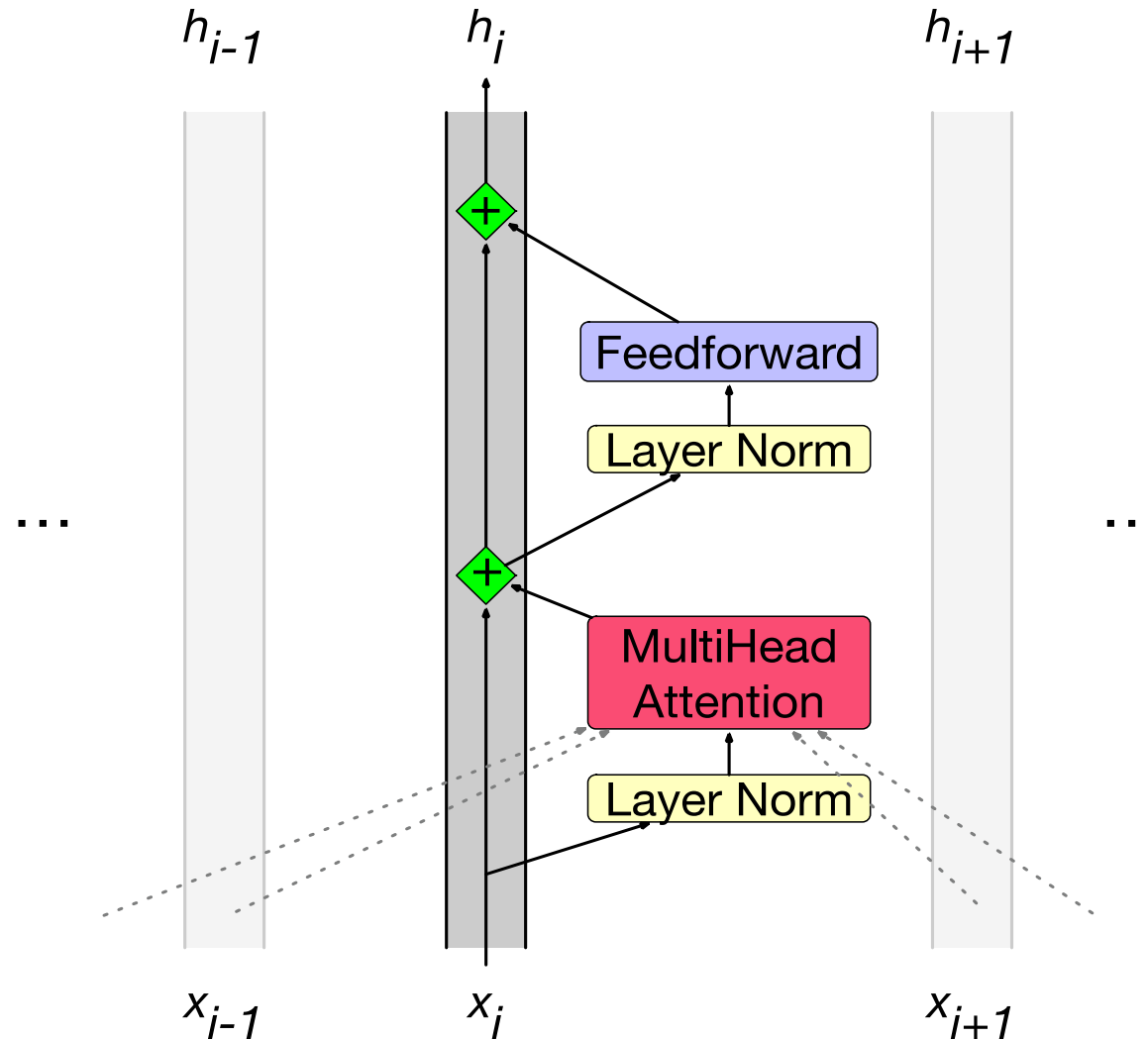


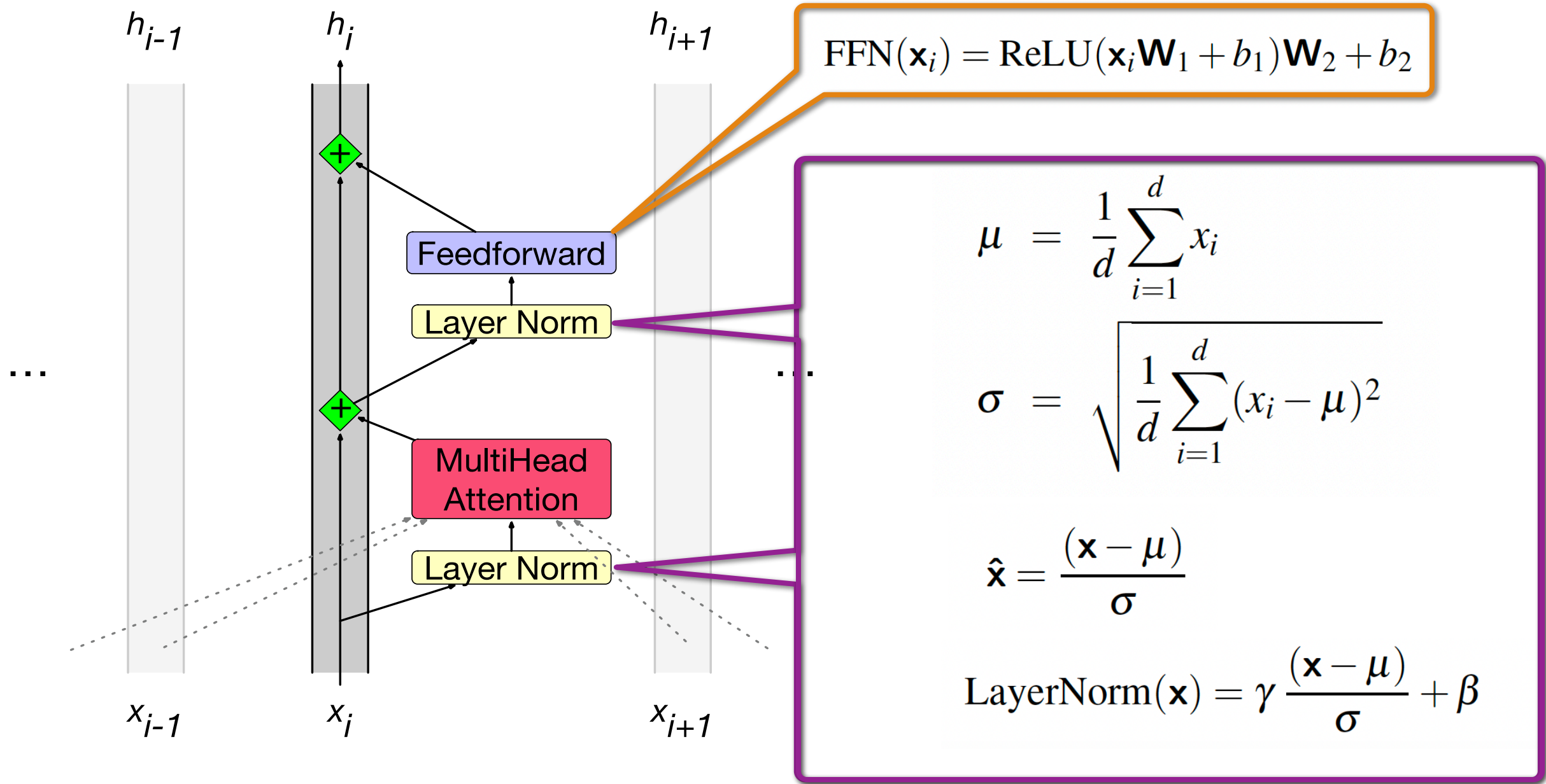
Transformers

The Transformer Block

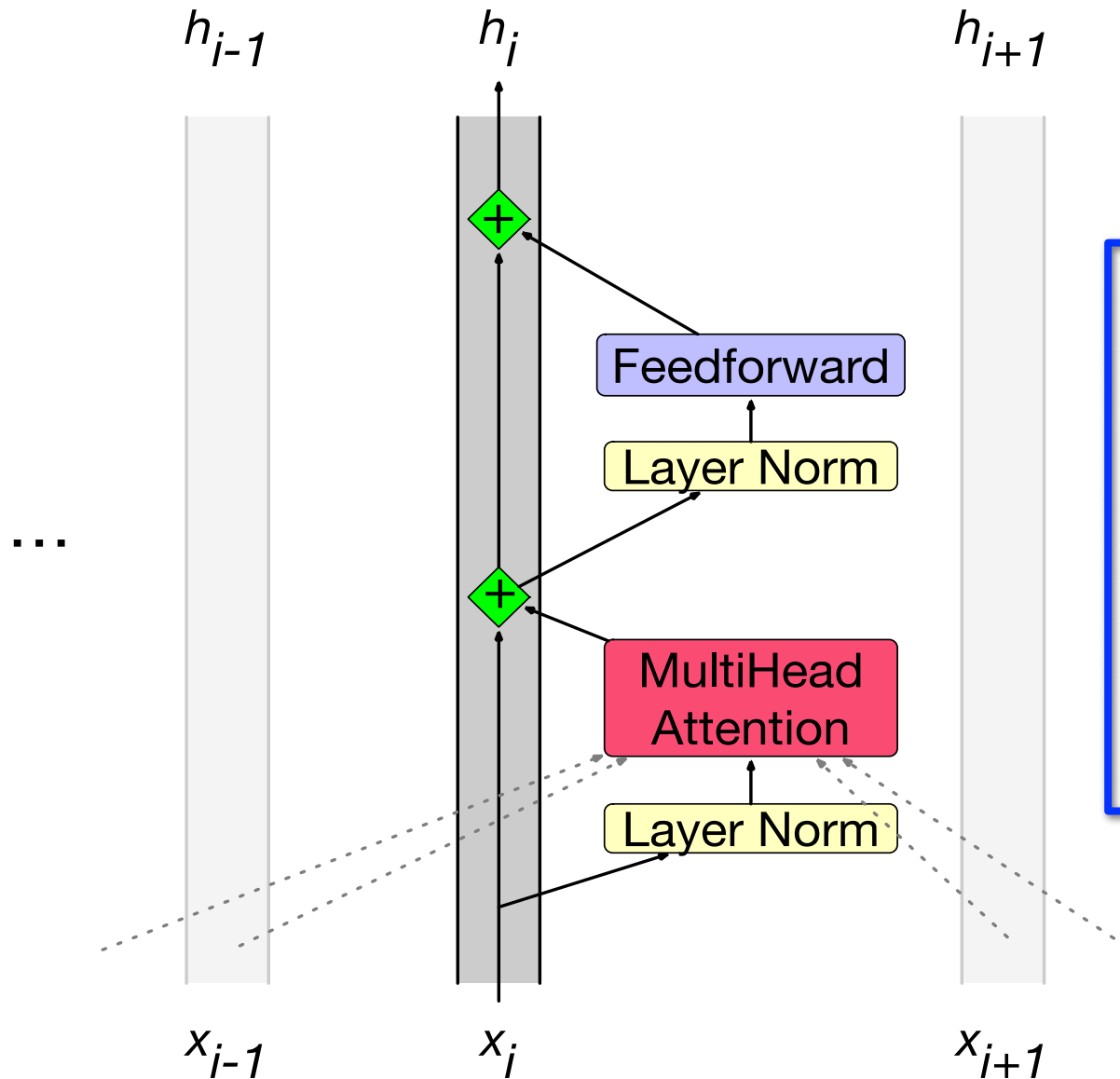


The residual stream: each token gets passed up and modified



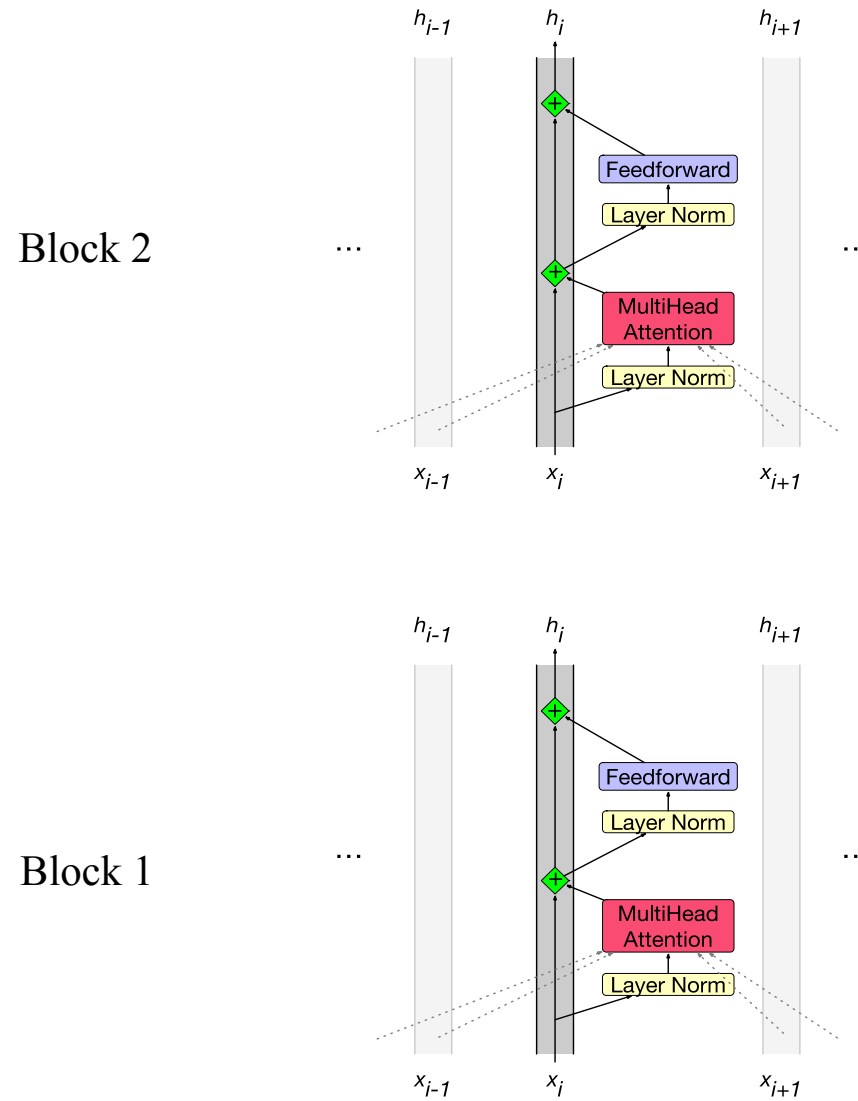


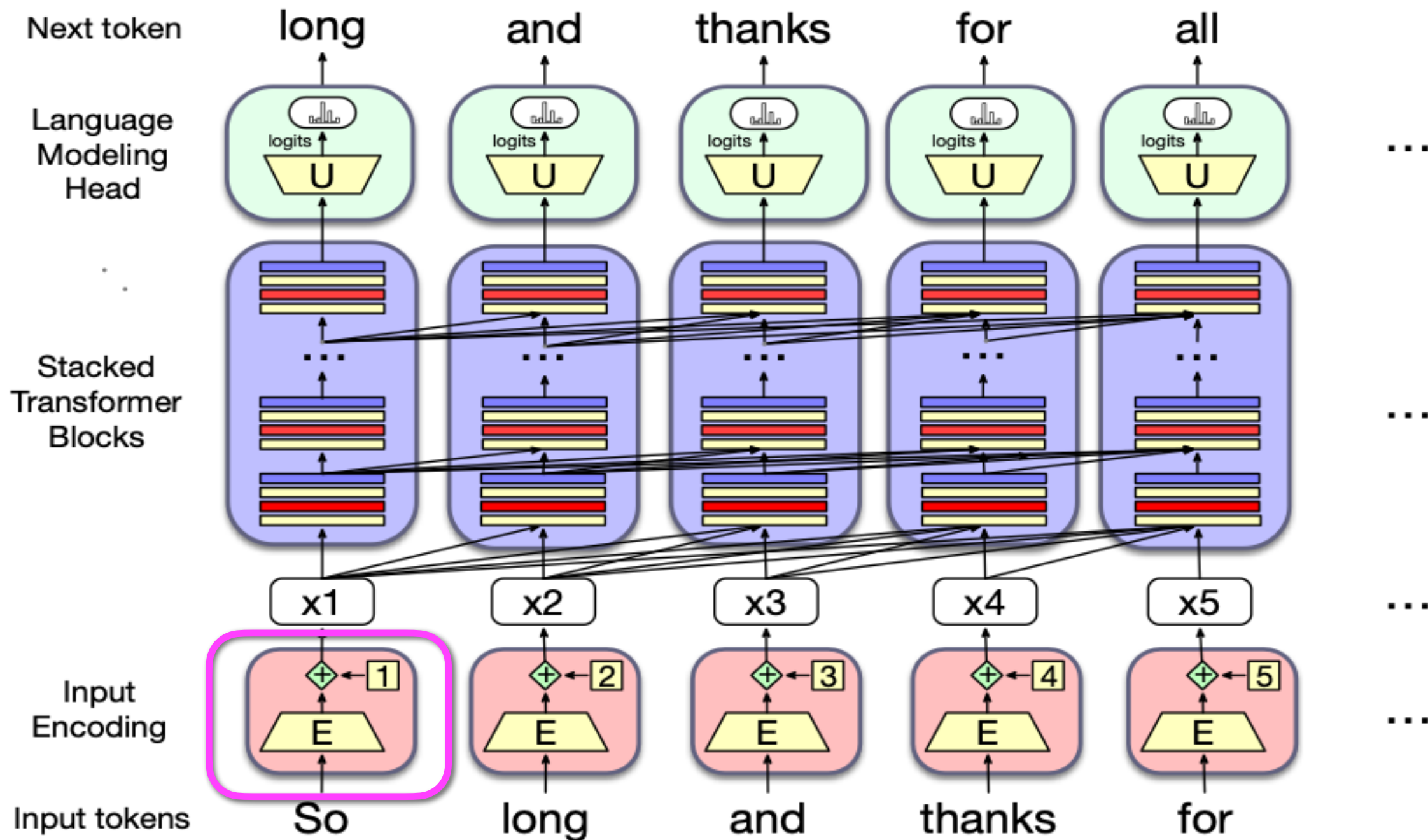
Putting together a single transformer block



$$\begin{aligned} \mathbf{t}_i^1 &= \text{LayerNorm}(\mathbf{x}_i) \\ \mathbf{t}_i^2 &= \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \dots, \mathbf{t}_N^1]) \\ \mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\ \mathbf{t}_i^4 &= \text{LayerNorm}(\mathbf{t}_i^3) \\ \mathbf{t}_i^5 &= \text{FFN}(\mathbf{t}_i^4) \\ \mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3 \end{aligned}$$

A transformer is a stack of these blocks





Token and Position Embeddings

The matrix X (of shape $[N \times d]$) has an embedding for each word in the context.

This embedding is created by adding two distinct embeddings for each input

- token embedding
- positional embedding

Token Embeddings

Embedding matrix E has shape $[|V| \times d]$.

- One row for each of the $|V|$ tokens in the vocabulary.
- Each word is a row vector of d dimensions

Given: string "*Thanks for all the*"

1. Tokenize and convert into vocab indices

$w = [5, 4000, 10532, 2224]$

2. Select the corresponding rows from E , each row an embedding

- (row 5, row 4000, row 10532, row 2224).

Position Embeddings

There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix ***E_{pos}*** of shape $[1 \times N]$.

Start with randomly initialized embeddings

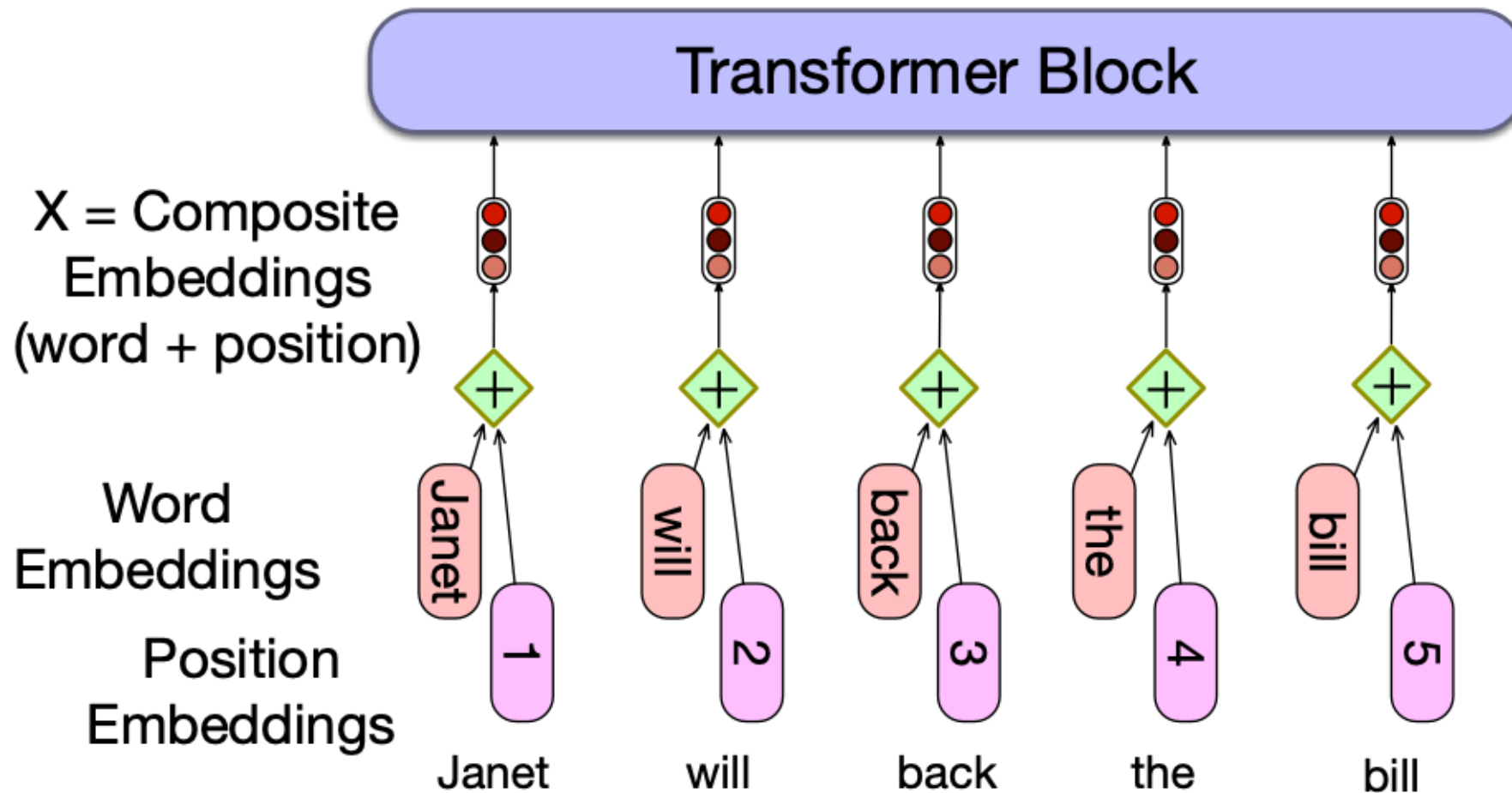
- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- As with word embeddings, these position embeddings are learned along with other parameters during training.

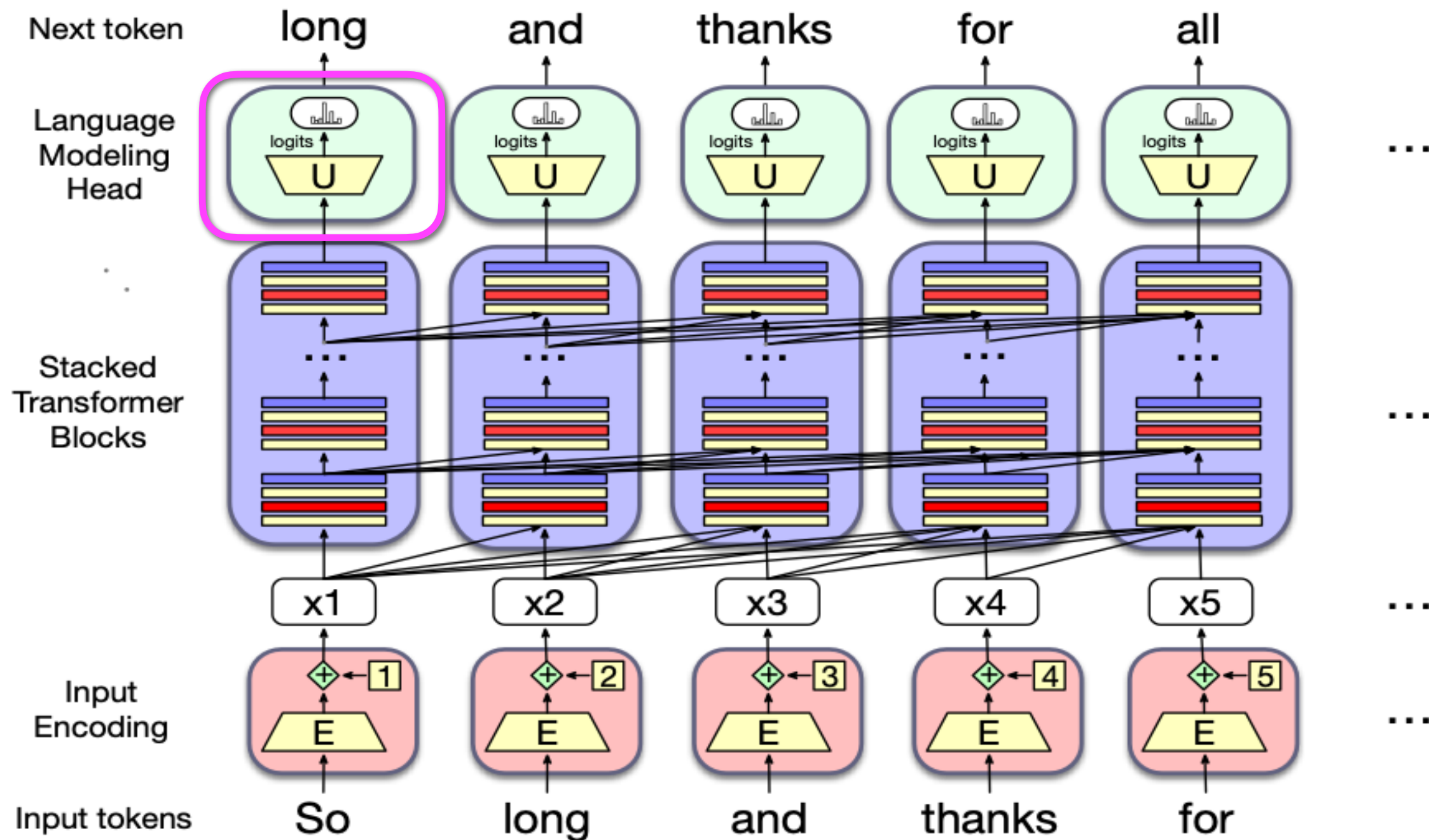
Position Embeddings

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Each x is just the sum of word and position embeddings

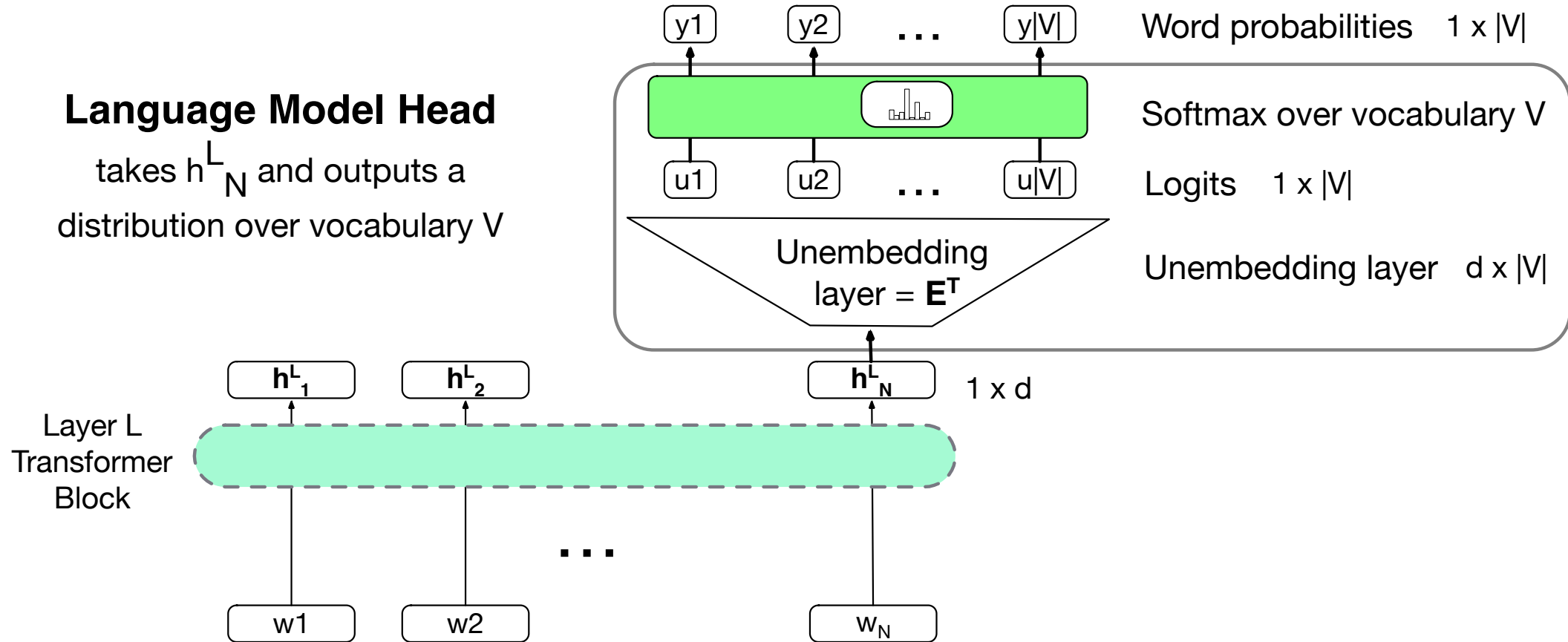




Language modeling head

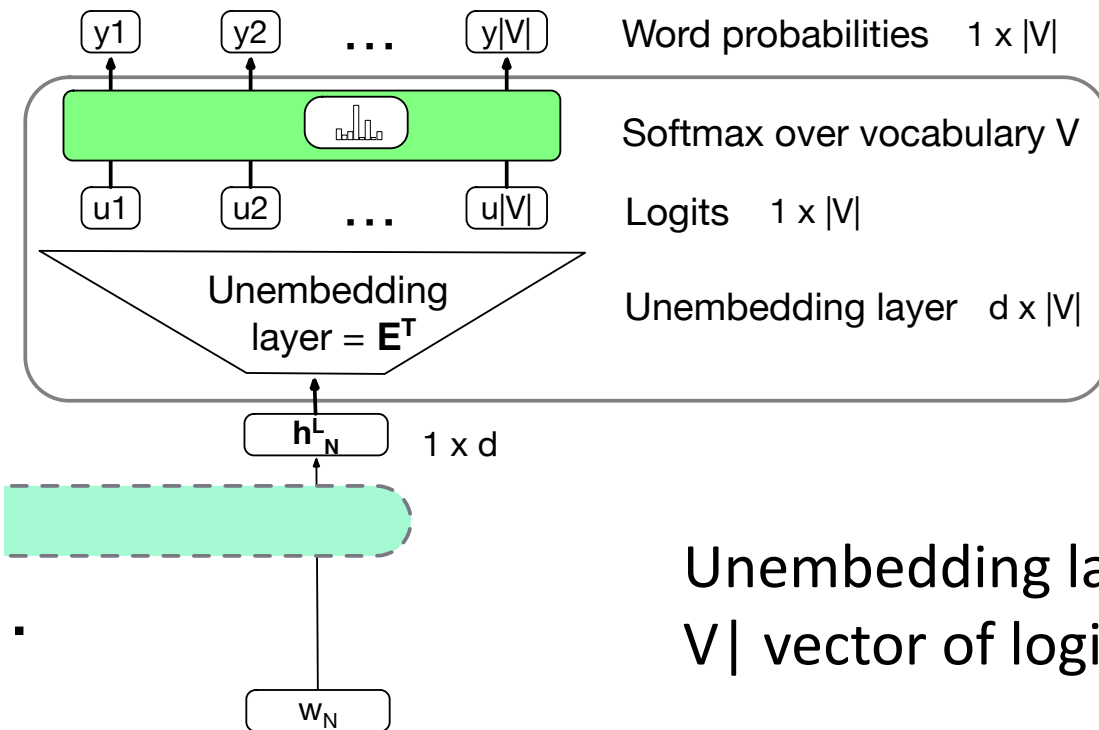
Language Model Head

takes h_N^L and outputs a distribution over vocabulary V



Language modeling head

Unembedding layer: linear layer projects from h_N^L (shape $[1 \times d]$) to logit vector



Why "unembedding"? **Tied** to E^T

Weight tying, we use the same weights for two different matrices

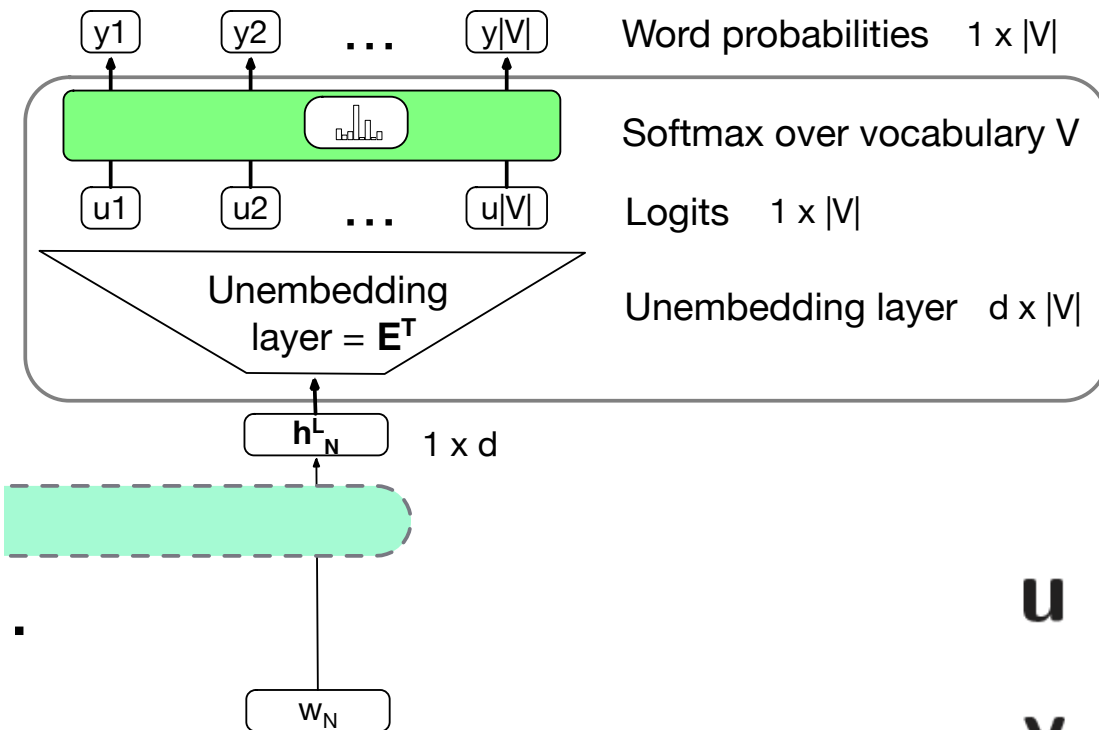
Unembedding layer maps from an embedding to a $1 \times |V|$ vector of logits

Language modeling head

Logits, the score vector $\mathbf{u}$

One score for each of the $|V|$ possible words in the vocabulary V .
Shape $1 \times |V|$.

Softmax turns the logits into probabilities over vocabulary.
Shape $1 \times |V|$.



$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$$

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

The final transformer model

